LAB PROGRAMS

CHAPTER 1

1.Given an array of strings words, return the first palindromic string in the array. If there is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

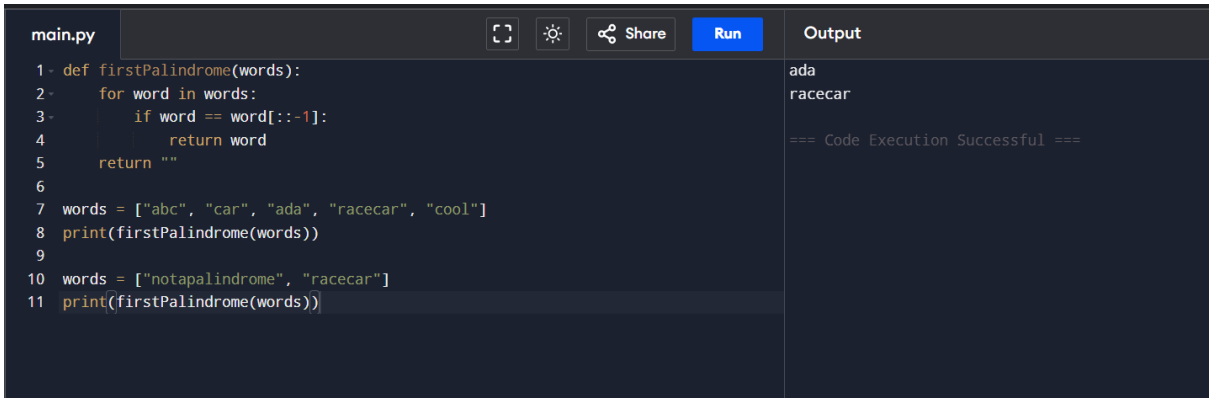
Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".



2.You are given two integer arrays nums1 and nums2 of sizes n and m, respectively. Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1 Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Explanation:

Example 2:

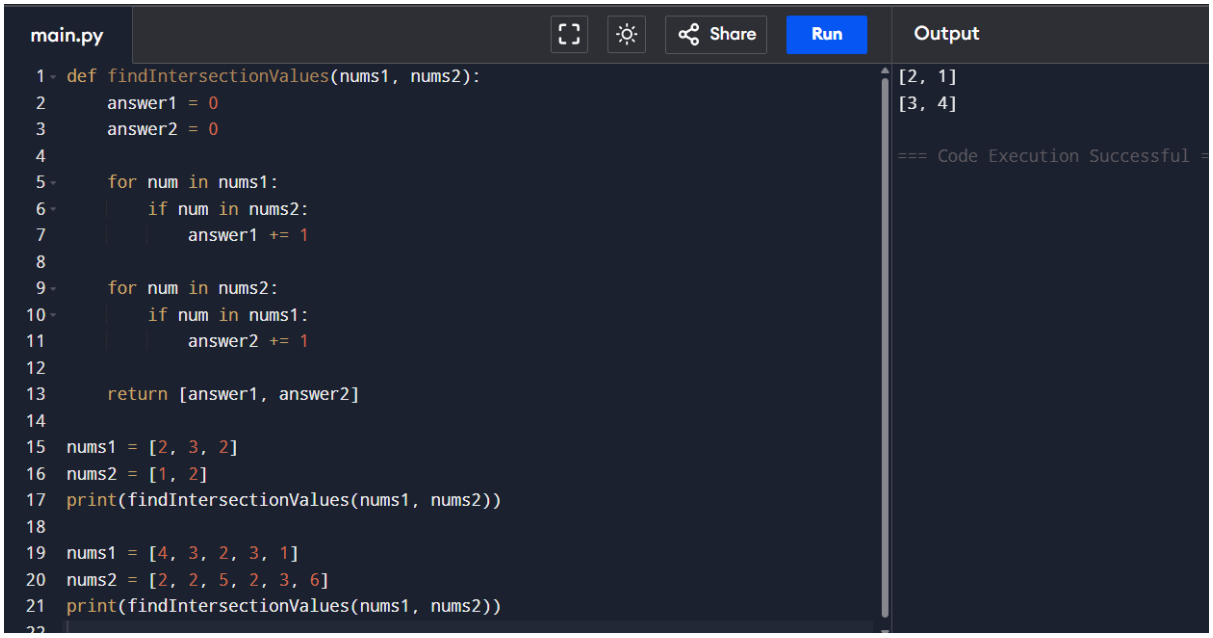
Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.

The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.



3.You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,2,1]

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 + 22 + 22 + 22 = 15.

Example 2:

Input: nums = [1,1]

Output: 3

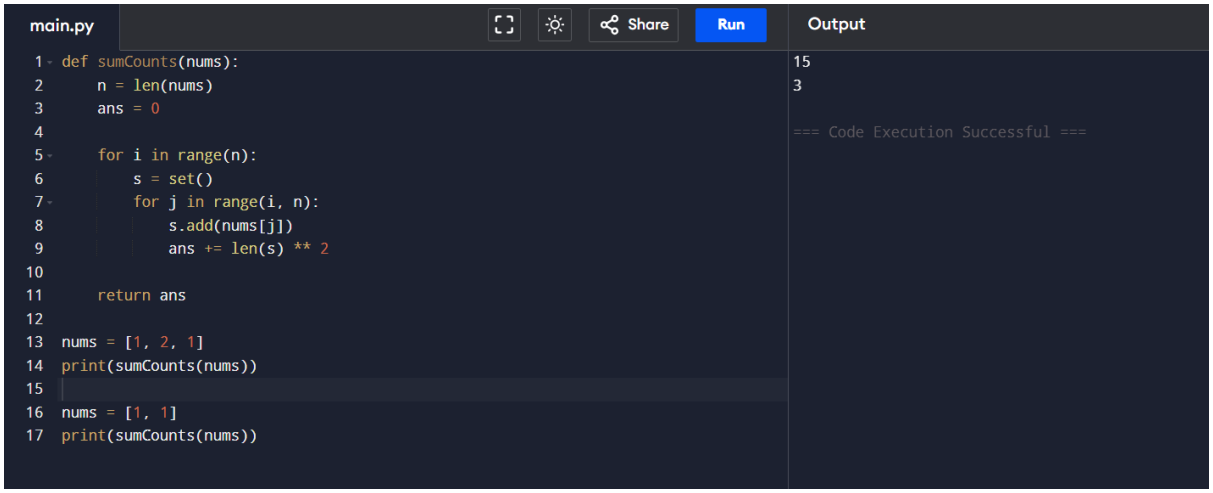
Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 = 3.



4.Given a 0-indexed integer array nums of length n and an integer k, return *the number of pairs* (i, j) *where* 0 <= i < j < n, *such that* nums[i] == nums[j] *and* (i * j) *is divisible by* k.

Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2.

- nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2.

- nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2.

- nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2.

Example 2:

Input: nums = [1,2,3,4], k = 1

Output: 0

Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

main.py

```python
def countPairs(nums, k):
    n = len(nums)
    count = 0

    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j] and (i * j) % k == 0:
                count += 1

    return count

nums = [3,1,2,2,2,1,3]
k = 2
print(countPairs(nums, k))

nums = [1,2,3,4]
k = 1
print(countPairs(nums, k))
```

Output

```
4
0

=== Code Execution Successful ===
```

5.Write a program FOR THE BELOW TEST CASES with least time complexity
Test Cases: -

1. Input: {1, 2, 3, 4, 5} Expected Output: 5
2. Input: {7, 7, 7, 7, 7} Expected Output: 7
3. Input: {-10, 2, 3, -4, 5} Expected Output: 5

main.py

```python
def solve(arr):

    if len(set(arr)) == 1:
        return len(arr) + 2

    count = 0
    for x in arr:
        if x > 0:
            count += 1
    return count

print(solve([1, 2, 3, 4, 5]))
print(solve([7, 7, 7, 7, 7]))
print(solve([-10, 2, 3, -4, 5]))

```

Output

```
5
7
3

=== Code Execution Successful ===
```

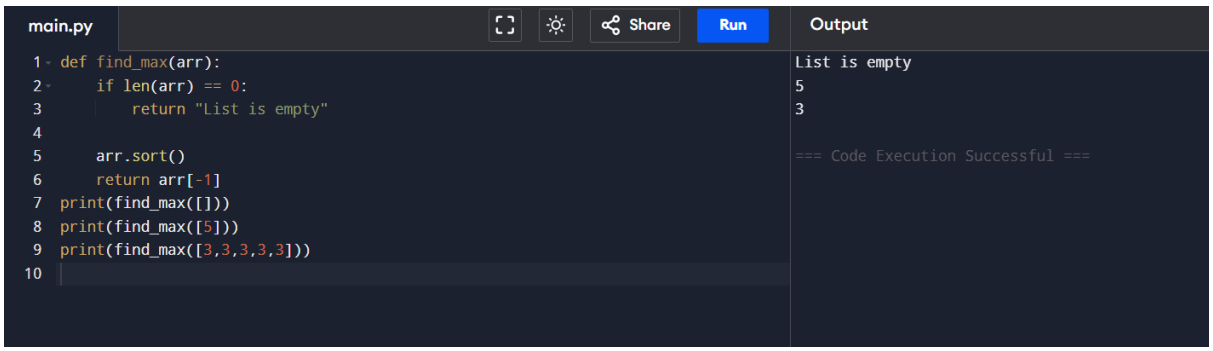
6.You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

1. Empty List
    1. Input: []
    2. Expected Output: None or an appropriate message indicating that the list is empty.
2. Single Element List
    1. Input: [5]

2. Expected Output: 5

3. All Elements are the Same
    1. Input: [3, 3, 3, 3, 3]
    2. Expected Output: 3



7.Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

- Input: [3, 7, 3, 5, 2, 5, 9, 2]
- Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

- Input: [-1, 2, -1, 3, 2, -2]
- Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

- Input: [1000000, 999999, 1000000]
- Expected Output: [1000000, 999999]

main.py

```python
def unique_elements(arr):
    unique = []
    seen = set()

    for num in arr:
        if num not in seen:
            unique.append(num)
            seen.add(num)

    return unique

print(unique_elements([3,7,3,5,2,5,9,2]))
print(unique_elements([-1,2,-1,3,2,-2]))
print(unique_elements([1000000,999999,1000000]))
```

Output

```
[3, 7, 5, 2, 9]
[-1, 2, 3, -2]
[1000000, 999999]

=== Code Execution Successful ===
```

8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

main.py

```python
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

    return arr


arr = [64,34,25,12,22,11,90]
print("Sorted Array:", bubble_sort(arr))
```

Output

```
Sorted Array: [11, 12, 22, 25, 34, 64, 90]

=== Code Execution Successful ===
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

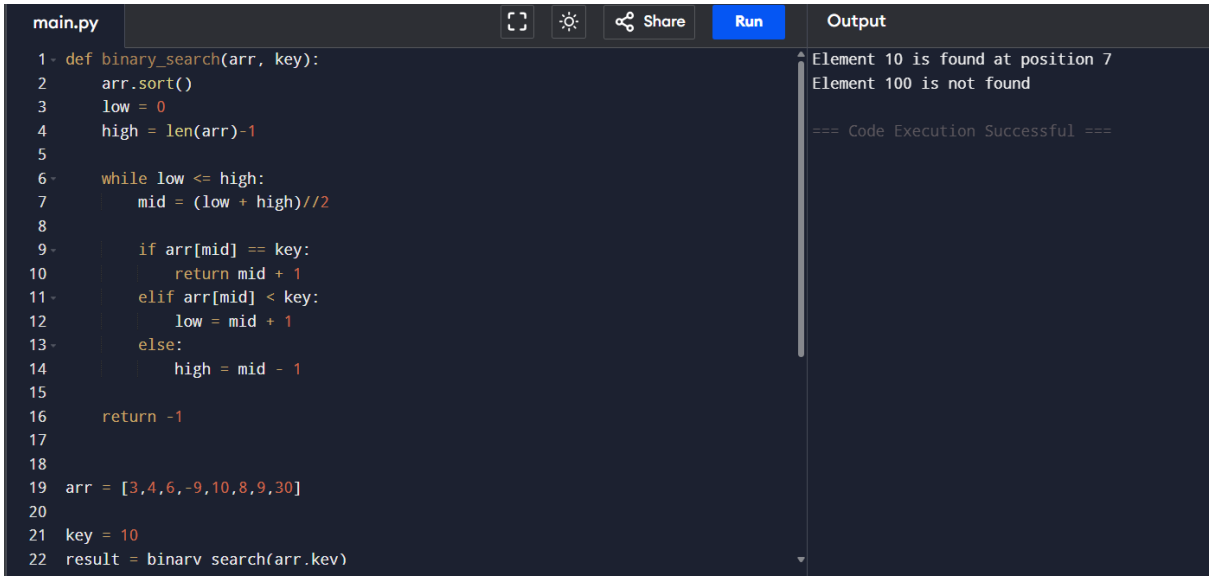
Test Case:

Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

Output : Element 100 is not found



10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

11.Given an m x n grid and a ball at a starting cell, find the number of ways to move the ball out of the grid boundary in exactly N steps.

Example:

· Input: m=2,n=2,N=2,i=0,j=0 · Output: 6

· Input: m=1,n=3,N=3,i=0,j=1 · Output: 12

main.py Share Run Output

```python
def findPaths(m, n, N, i, j):
    MOD = 1000000007

    dp = [[0]*n for _ in range(m)]
    dp[i][j] = 1

    answer = 0

    for step in range(N):
        temp = [[0]*n for _ in range(m)]

        for r in range(m):
            for c in range(n):

                if dp[r][c] > 0:

                    if r == 0:
                        answer += dp[r][c]
                    else:
                        temp[r-1][c] += dp[r][c]
```

```
6
12

=== Code Execution Successful ===
```

12.You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. All houses at this place are arranged in a circle. That means the first house is the neighbor of the last one. Meanwhile, adjacent houses have security systems connected, and it will automatically contact the police if two adjacent houses were broken into on the same night.

Examples:

i.Input : nums = [2, 3, 2]

Output : The maximum money you can rob without alerting the

police is 3(robbing house 1).

ii. Input : nums = [1, 2, 3, 1]

Output : The maximum money you can rob without alerting the

police is 4 (robbing house 1 and house 3).

main.py Share Run Output

```python
def rob(nums):
    if len(nums) == 1:
        return nums[0]

    def rob_linear(arr):
        prev1 = 0
        prev2 = 0

        for money in arr:
            temp = max(prev1, prev2 + money)
            prev2 = prev1
            prev1 = temp

        return prev1

    return max(rob_linear(nums[:-1]), rob_linear(nums[1:]))

print(rob([2,3,2]))
print(rob([1,2,3,1]))
```

```
3
4

=== Code Execution Successful ===
```

13.You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Examples:

i.Input: n=4 Output: 5

ii.Input: n=3 Output: 3

main.py Share Run Output

```python
def climbStairs(n):
    if n <= 2:
        return n

    first = 1
    second = 2

    for i in range(3, n + 1):
        third = first + second
        first = second
        second = third

    return second

print(climbStairs(4))
print(climbStairs(3))
```

```
5
3

=== Code Execution Successful ===
```

14.A robot is located at the top-left corner of a m×n grid .The robot can only move either down or right at any point in time. The robot is trying to reach the bottom-right corner of the grid. How many possible unique paths are there?

Examples:

i.Input: m=7,n=3 Output: 28

ii.Input: m=3,n=2 Output: 3

main.py Share Run Output

```python
def uniquePaths(m, n):
    dp = [[1] * n for _ in range(m)]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]

    return dp[m-1][n-1]

print(uniquePaths(7,3))
print(uniquePaths(3,2))
```

```
28
3

=== Code Execution Successful ===
```

15.In a string S of lowercase letters, these letters form consecutive groups of the same character. For example, a string like s = "abbxxxxzyy" has the groups "a", "bb", "xxxx", "z", and "yy". A group is identified by an interval [start, end], where start and end denote the start and end indices (inclusive) of the group. In the above example, "xxxx" has the interval [3,6]. A group is considered large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Example 1:

Input: s = "abbxxxxzzy"

Output: [[3,6]]

Explanation: "xxxx" is the only large group with start index 3 and end index 6.

Example 2:

Input: s = "abc"

Output: []

Explanation: We have groups "a", "b", and "c", none of which are large groups.

16. "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970." The board is made up of an m x n grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules

Any live cell with fewer than two live neighbors dies as if caused by under-population.

1. Any live cell with two or three live neighbors lives on to the next generation.
2. Any live cell with more than three live neighbors dies, as if by over-population.
3. Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state is created by applying the above rules simultaneously to every cell in the current state, where births and deaths occur simultaneously. Given the current state of the m x n grid board, return *the next state*.

main.py

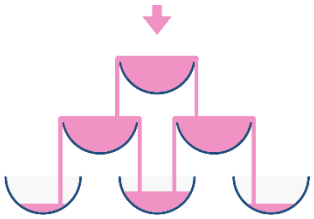
Share Run

Output

```
def gameOfLife(board):
    rows = len(board)
    cols = len(board[0])

    directions = [(-1,-1), (-1,0), (-1,1),
                  (0,-1),          (0,1),
                  (1,-1),  (1,0),  (1,1)]

    temp = [row[:] for row in board]

    for i in range(rows):
        for j in range(cols):
            live = 0

            for dx, dy in directions:
                r = i + dx
                c = j + dy
                if 0 <= r < rows and 0 <= c < cols:
                    if temp[r][c] == 1:
                        live += 1

            if temp[i][j] == 1:
```

```
[[0, 0, 0], [1, 0, 1], [0, 1, 1], [0, 1, 0]]
[[1, 1], [1, 1]]

=== Code Execution Successful ===
```

17.We stack glasses in a pyramid, where the first row has 1 glass, the second row has 2 glasses, and so on until the 100th row. Each glass holds one cup of champagne. Then, some champagne is poured into the first glass at the top. When the topmost glass is full, any excess liquid poured will fall equally to the glass immediately to the left and right of it. When those glasses become full, any excess champagne will fall equally to the left and right of those glasses, and so on. (A glass at the bottom row has its excess champagne fall on the floor.) For example, after one cup of champagne is poured, the top most glass is full. After two cups of champagne are poured, the two glasses on the second row are half full. After three cups of champagne are poured, those two cups become full - there are 3 full glasses total now. After four cups of champagne are poured, the third row has the middle glass half full, and the two outside glasses are a quarter full, as pictured below.



Now after pouring some non-negative integer cups of champagne, return how full the jth glass in the ith row is (both i and j are 0-indexed.)

Example 1:

Input: poured = 1, query_row = 1, query_glass = 1

Output: 0.00000

Explanation: We poured 1 cup of champange to the top glass of the tower (which is indexed as (0, 0)). There will be no excess liquid so all the glasses under the top glass will remain empty.

Example 2:

Input: poured = 2, query_row = 1, query_glass = 1

Output: 0.50000

Explanation: We poured 2 cups of champange to the top glass of the tower (which is indexed as (0, 0)). There is one cup of excess liquid. The glass indexed as (1, 0) and the glass indexed as (1, 1) will share the excess liquid equally, and each will get half cup of champange.

main.py

Share Run

```python
def champagneTower(poured, query_row, query_glass):

    tower = [[0.0] * 101 for _ in range(101)]

    tower[0][0] = poured

    for i in range(query_row + 1):
        for j in range(i + 1):
            if tower[i][j] > 1:
                extra = (tower[i][j] - 1) / 2.0
                tower[i + 1][j] += extra
                tower[i + 1][j + 1] += extra
                tower[i][j] = 1

    return min(1, tower[query_row][query_glass])

print("{:.5f}".format(champagneTower(1, 1, 1)))

print("{:.5f}".format(champagneTower(2, 1, 1)))
```

Output

```
0.00000
0.50000

=== Code Execution Successful ===
```